

Task 3 —: Create an IAM User and Apply Least Privilege

Intern: Nidhi Pandey

Program: Evolixa AWS Cloud Internship

Date: 2026-09-05

1. Approach and Architecture

The main goal of this task was to understand how IAM identities and permissions work by creating a controlled access scenario. The scenario was about a junior developer who needs to upload and manage files in a specific S3 bucket, but should not have unrestricted access to the entire AWS account.

Instead of directly creating the least privilege policy, I first explored the "AdministratorAccess" policy to understand what "too much access" looks like. Then, I designed a least-privilege policy that strictly limits the junior developer to only the S3 actions (ListBucket, GetObject, PutObject) required for their job, and restricts those actions to a specific bucket (task2-nidhi-website). I tested both permitted and denied operations to confirm the policies worked correctly. I also did a practical test by creating a test-user with a S3ReadOnlyPolicy to demonstrate an allowed (GetObject) and a denied (PutObject) operation.

2. Implementation Steps

Step	Action	Details
1	Create a test IAM identity	Created task3-iam-user with programmatic access only.
2	Identify required access	Identified the developer needed S3: List, Get, Put, Delete.
3	Create/use an IAM policy	Attached AdministratorAccess initially (for learning). Later, created a custom restricted policy.
4	Restrict permissions to required S3 actions	Used S3ReadOnlyPolicy (allowing only s3:GetObject) to test denied actions.
5	Restrict access to specific bucket	Restricted policy resource to arn:aws:s3:::task2-nidhi-website/*.
6	Test an allowed operation	Used aws s3 cp s3://task2-nidhi-website/index.html . (Allowed).
7	Test a denied operation	Used aws s3 cp test.txt s3://task2-nidhi-website/ (Denied -> AccessDenied).
8	Record the results	Documented the outcomes in this report.
9	Explain admin risks	Noted why broad permissions are risky (security risk, no audit trail).
10	Review unnecessary access	Confirmed AdminAccess was unnecessary for this scenario.
11	Document final access model	Created a final IAM access matrix for the developer role.

12	Explain principle of least privilege	Documented the concept in the README and this report.
13	Identify production improvements	Suggested using IAM roles, MFA, and CloudTrail.
14	Remove test resources	Deleted test-user and S3ReadOnlyPolicy after testing.
15	Document security considerations	Added security considerations in the final documentation.

3. Key AWS Concepts Learned

- **IAM Users:** An identity with long-term credentials used to interact with AWS.
- **IAM Policies:** JSON documents that define permissions (Allow/Deny).
- **Least Privilege:** Granting only the permissions required to perform a specific job, nothing more.
- **Programmatic Access:** Using Access Keys and Secret Keys for CLI/SDK access.
- **Resource-Based vs Identity-Based Policies:** Understanding how bucket policies differ from IAM user policies.
- **Testing Permissions:** Using the AWS CLI to simulate both Allowed and Denied API calls.

4. Challenges Faced

1. **Initial Over-Permissioning:** The first instinct was to simply attach AdministratorAccess to get the task done quickly. It worked, but it violated the core objective of the task.

2. Understanding Denied

Actions: Since AdministratorAccess allowed everything, I had to create a separate restricted user (test-user) and a specific policy (S3ReadOnlyPolicy) to genuinely test a Denied operation.

3. **CLI Profile Switching:** Initially, I forgot to use the --profile test-user flag while testing, so my commands were running with the s3-admin admin permissions, bypassing the restrictions I wanted to test.
4. **Permission Propagation:** Sometimes, after attaching a policy, it takes a few seconds for the permissions to propagate fully, leading to temporary "AccessDenied" errors.

5. Solutions Implemented

1. **Least Privilege Design:** Instead of AdministratorAccess, I designed a custom policy that only allowed s3:ListBucket, s3:GetObject, s3:PutObject, and s3:DeleteObject on a specific bucket ARN.
2. **Denied Test Setup:** I created S3ReadOnlyPolicy (allowing only s3:GetObject) and attached it to a new test-user. This clearly proved that s3:PutObject was denied.
3. **Profile Management:** I consistently used --profile test-user in my CLI commands to ensure I was testing the right user's permissions.
4. **Testing Strategy:** I always waited 5-10 seconds after attaching policies and used aws sts get-caller-identity to confirm which user/profile was currently active.

6. Security Considerations

- **Principle of Least Privilege:** The junior developer's access was strictly limited to S3 actions on a specific bucket.

- **No Hardcoded Credentials:** Access keys were generated but stored securely (not in the code). I used AWS CLI profiles to manage them safely.
 - **Restricted Actions:** The policy explicitly denies any ec2, iam, or rds actions, preventing the user from accidentally altering infrastructure.
 - **Audit Trail:** If CloudTrail were enabled, all actions would be logged, providing accountability.
 - **MFA:** While not implemented in this test environment, MFA was recommended for production to protect against credential theft.
-

7. Results and Observations

Allowed Operation (GetObject)

bash

```
aws s3 cp s3://task2-nidhi-website/index.html ./ --profile test-user
```

Result: download: s3://task2-nidhi-website/index.html to ./index.html (Success).

Denied Operation (PutObject)

bash

```
aws s3 cp test.txt s3://task2-nidhi-website/ --profile test-user
```

Result: An error occurred (AccessDenied) when calling the PutObject operation: User: arn:aws:iam::...:user/test-user is not authorized to perform: s3:PutObject (Access Denied).

Observations

- IAM policies are highly granular. They can control specific actions on specific resources.
- The AdministratorAccess is powerful but dangerous for junior developers.
- The CLI is an excellent tool for quickly verifying IAM policies without using the AWS console.

8. Possible Improvements for Production

Improvement	Benefit
Use IAM Roles instead of Access Keys	Avoids the risk of key leakage. Use roles for EC2 or other services.
Enable MFA (Multi-Factor Authentication)	Adds an extra layer of security to prevent unauthorized access even if credentials are compromised.
Implement AWS Organizations SCPs	Add guardrails at the organizational level to prevent users from exceeding limits even if policies allow it.
Enable CloudTrail Data Events	Log all S3 data plane events (GetObject, PutObject) for audit compliance.
Regular Permission Reviews	Quarterly review of all IAM policies to remove unnecessary permissions (reduce the attack surface).
Use IAM Access Analyzer	Continuously monitor policies for public access or over-permissioning.

Conclusion

This task provided hands-on experience in creating IAM users, designing least-privilege policies, and testing both allowed and denied operations. The distinction between broad administrative permissions and least-privilege access is critical in cloud security. By enforcing strict resource-based and action-based restrictions, organizations can significantly reduce the risk of data breaches and accidental system modifications.